

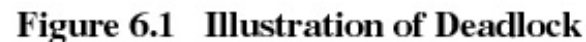
Sistemas Operacionais

Deadlock

Conteúdo

- Deadlock e recursos
- Modelagem de deadlock
- Detecção de deadlock
- Recuperação de deadlock
- Evitando deadlock
- Prevenção de deadlock

- Envolve necessidade conflitante por recursos entre dois ou mais processos



Recursos

- Deadlock pode ocorrer quando processos têm acesso exclusivo a recursos do sistema
- Recursos podem ser:
 - Um **dispositivo de hardware** → impressora, fita magnética, etc
 - Uma **informação** → registro travado em um banco de dados, arquivo, etc
- Em geral, um recursos pode ser composto por várias instâncias (ex.: HD, impressora, etc)
- Um recurso pode ser utilizado por apenas um processo por vez

Recursos (cont.)

- Processos com acesso a recursos negados são colocados em estado de bloqueio
- A ordem em que os recursos são requeridos é importante

```
typedef int semaphore;  
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

sem deadlock

```
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources( );  
    up(&resource_1);  
    up(&resource_2);  
}
```

deadlock em potencial

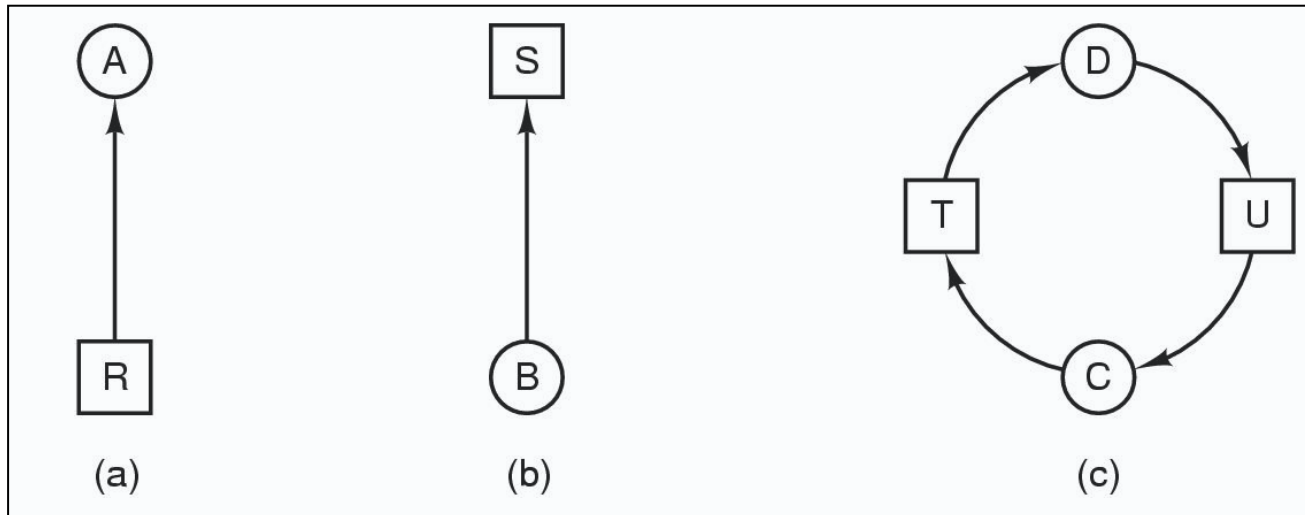
- 1- Processo_A obtém rec_1
- 2- Processo_B obtém rec_2
- 3- proc_A bloq. por rec_2
- 4- proc_B bloq. Por rec_1
- 5- deadlock ...

DEADLOCK: definição formal

- “Um conjunto de processos está em estado de *deadlock* se cada um de seus processos está esperando por um evento que somente outro processo no conjunto pode causar”
- Condições para ocorrência de deadlock
 - **Exclusão mútua:** somente um processo por vez pode usar um recurso
 - **Posse e espera:** processos já de posse de recursos podem requerer novos recursos
 - **Sem preempção:** nenhum processo pode ser forçado a liberar um recurso, mas deve fazê-lo voluntariamente
 - **Espera circular:** há uma cadeia circular de 2 ou mais processos, os quais esperam pelo recurso do processo seguinte na cadeia

DEADLOCK: modelagem

- Uso da teoria dos grafos



- a) Recurso R atribuído ao processo A
- b) Processo B requerendo/esperando pelo recurso S
- c) Processo C e D em deadlock sobre os recursos T e U

DEADLOCK: modelagem (cont.)

A
Request R
Request S
Release R
Release S

(a)

B
Request S
Request T
Release S
Release T

(b)

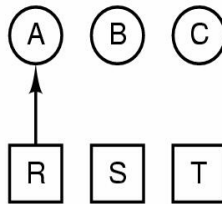
C
Request T
Request R
Release T
Release R

(c)

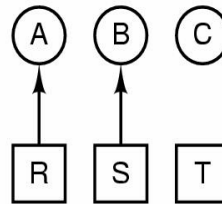
Escalonador:
Circular (round-robin) !

1. A requests R
2. B requests S
3. C requests T
4. A requests S
5. B requests T
6. C requests R
deadlock

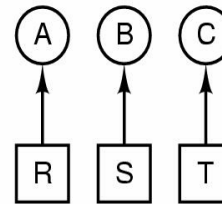
(d)



(e)

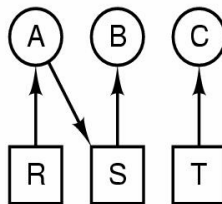


(f)

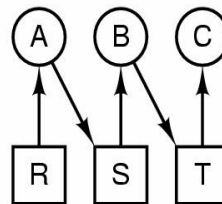


(g)

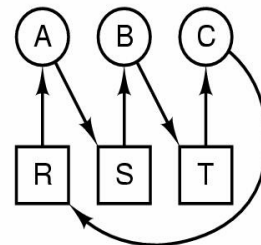
A ordem de execução
dos processos leva o
sistema a uma espera
circular → deadlock



(h)



(i)

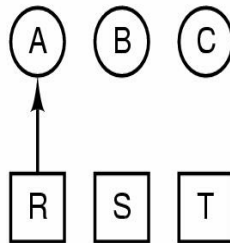


(j)

DEADLOCK: modelagem (cont.)

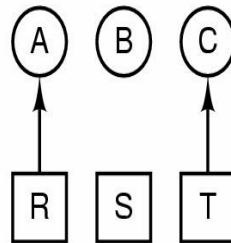
Mudando a ordem de execução dos processos previne o deadlock !!!

1. A requests R
 2. C requests T
 3. A requests S
 4. C requests R
 5. A releases R
 6. A releases S
- no deadlock

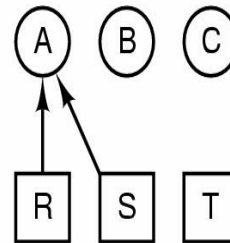


(k)

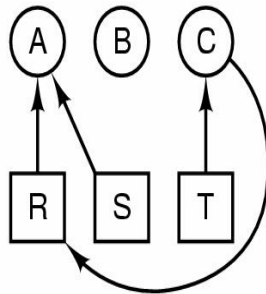
(l)



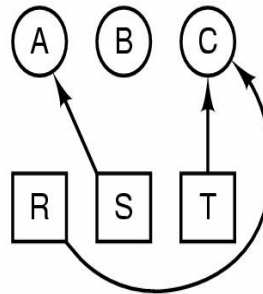
(m)



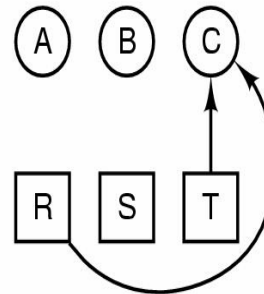
(n)



(o)



(p)



(q)

DEADLOCK: detecção c/ múltiplos recursos p/ tipo

Vetor de recursos existentes

Resources in existence
($E_1, E_2, E_3, \dots, E_m$)

Matriz de alocação atual

Current allocation matrix

C_{11}	C_{12}	C_{13}	$\dots$	C_{1m}
C_{21}	C_{22}	C_{23}	$\dots$	C_{2m}
$\vdots$	$\vdots$	$\vdots$		$\vdots$
C_{n1}	C_{n2}	C_{n3}	$\dots$	C_{nm}

Row n is current allocation to process n

Resources available
($A_1, A_2, A_3, \dots, A_m$)

Vetor de recursos disponíveis

Request matrix

R_{11}	R_{12}	R_{13}	$\dots$	R_{1m}
R_{21}	R_{22}	R_{23}	$\dots$	R_{2m}
$\vdots$	$\vdots$	$\vdots$		$\vdots$
R_{n1}	R_{n2}	R_{n3}	$\dots$	R_{nm}

Matriz de pedido

Row 2 is what process 2 needs

$m \rightarrow$ nr. de recursos
 $n \rightarrow$ nr. de proc

■ Algoritmo baseado em matriz

1. Procure por um processo desmarcado, P_i , para o qual a i -ésima linha da matriz R é menor ou igual a A
2. Se encontrado, adicione a i -ésima linha de C a A , marque o processo e retorne ao procedimento 1
3. Se não existe tal processo, o algoritmo termina

→ Quando o algoritmo termina, todos os processos desmarcados estão em deadlock

DEADLOCK: exemplo de detecção

- 3 processos e 4 classes de recursos

Execução do algoritmo

- Verificar linha a linha da matriz R
- P1 não pode ser atendido: 0 CD-Rom disponível
- P2 também não: nenhum scanner livre
- P3 é atendido, roda e finalmente retorna todos seus recursos
→ $A=(2\ 2\ 2\ 0)$
- P2 pode então rodar e retornar
→ $A=(4\ 2\ 2\ 1)$
- O processo remanescente pode então rodar → SEM DEADLOCK

	Tape drives	Plotters	Scanners	CD Roms
$E =$	4	2	3	1
$A =$	2	1	0	0

P1 → tem um scanner

P2 → tem 2 TDs e um CD-Rom

P3 → tem 1 plotter e 2 scanners

Current allocation matrix

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix} \begin{matrix} P1 \\ P2 \\ P3 \end{matrix}$$

Request matrix

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Cada processo precisa de recursos adicionais !

DEADLOCK: exercício

- Considerando-se o exemplo anterior, como ficaria a situação se o processo P2 necessitasse dos recursos abaixo?
 - 1 CD-ROM
 - 2 TDs
 - 1 plotter

DEADLOCK: aplicabilidade da detecção

Quando a detecção deve ser realizada?

- Sempre que um recurso for solicitado
 - rápido, mas dispendioso em termos de uso de CPU
- Periodicamente
 - lento
- Somente quando o uso da CPU cair abaixo de um certo limite
 - quando alguns processos se encontram em deadlock, provavelmente poucos processos estão em execução e assim a CPU deve estar bem ociosa

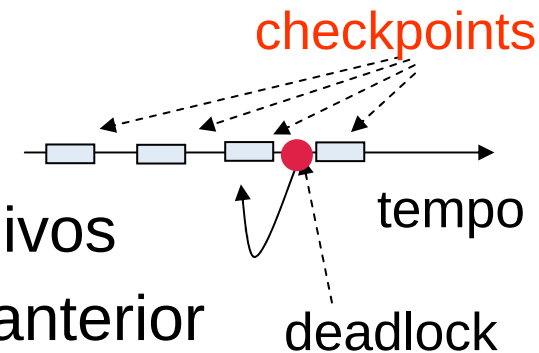
Recuperação de deadlock

■ Por preempção

- Retire o recurso de algum outro processo
- Dificuldade: nem sempre é viável para se implementar

■ Por retrocesso a checkpoints

- A medida que um processo roda, seus dados são armazenados em arquivos
- Sob deadlock, o ponto imediatamente anterior ao uso do recurso causando deadlock é retornado

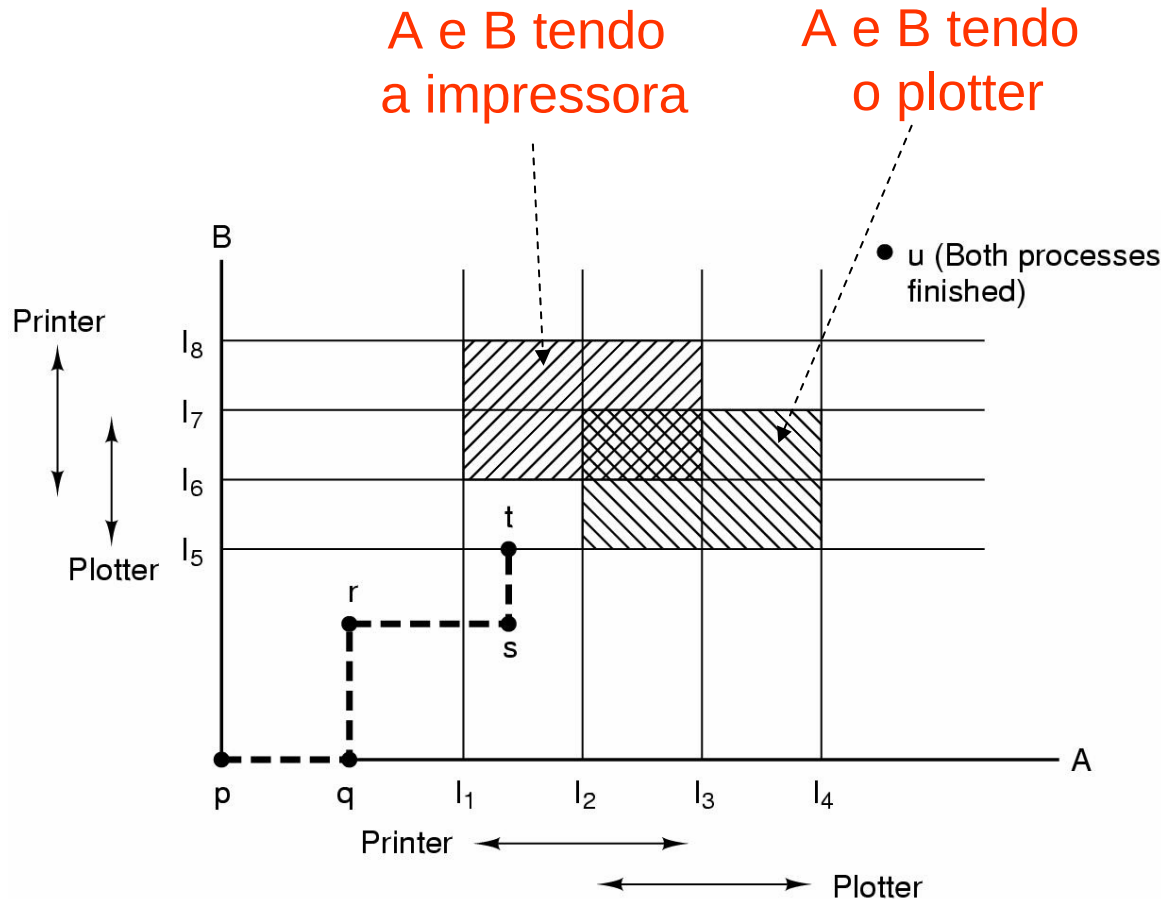


■ Por destruição de processos

- Um processo no loop de deadlock ou outro processo que possua um recurso necessário ao loop
- Cautela na escolha do processo a ser destruído

Evitando deadlocks: trajetória de recursos

- Evita-se deadlocks tendo-se certas info de antemão, alocando-se recursos apropriadamente
- No ponto t, B solicita o plotter, se receber o sistema irá a deadlock
- A região riscada (escura) é insegura
- A exclusão mútua evita que se entre nessa região
- O mais sensato em t, é bloquear B até que A esteja em l4
- Depois de l4, u pode ser alcançado sem deadlock



Evitando deadlocks: estados (in)seguro

O estado (a) é seguro !

Has Max		
A	3	9
B	2	4
C	2	7

Free: 3

(a)

$$3+2+2=7 \rightarrow 10-7=3$$

Has Max		
A	3	9
B	4	4
C	2	7

Free: 1

(b)

Has Max		
A	3	9
B	0	—
C	2	7

Free: 5

(c)

Has Max		
A	3	9
B	0	—
C	7	7

Free: 0

(d)

Has Max		
A	3	9
B	0	—
C	0	—

Free: 7

(e)

Neste exemplo, há um total de 10 instâncias do recurso em questão

Estado seguro

- Não há deadlock e existe uma ordem de escalonamento em que todos os processos podem rodar completamente ainda que precisem de todos os seus recursos imediatamente

Estado inseguro

- Há um deadlock potencial que ocorrerá se todos os recursos de cada processo forem necessários

Evitando deadlocks: estados (in)seguro (cont.)

A e C precisam de 5 recursos, mas só existem 4 !!!

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

(a)

	Has	Max
A	4	9
B	2	4
C	2	7

Free: 2

(b)

	Has	Max
A	4	9
B	4	4
C	2	7

Free: 0

(c)

	Has	Max
A	4	9
B	—	—
C	2	7

Free: 4

(d)

O estado (b) é inseguro !

A recebe mais um recurso

B recebe mais dois recursos

B termina e libera seus recursos

- A ordem de escalonamento pode ser importante
- No exemplo acima, A não deveria ter recebido o recurso adicional
- Estado inseguro não é sinônimo de deadlock

Evitando deadlocks: Banker's algoritmo

seguro

Has Max

A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

seguro

Has Max

A	1	6
B	1	5
C	2	4
D	4	7

Free: 2

(b)

inseguro

Has Max

A	1	6
B	2	5
C	2	4
D	4	7

Free: 1

(c)

- Modelo baseado num sistema bancário de uma cidade pequena
- Clientes → processos, unidades → recursos, banq. → sistema oper
- Este algoritmo verifica cada solicitação por recurso, e somente atende a solicitação se a mesma induzir o sistema a um estado seguro
- Solicitação não atendidas são colocadas em espera

Exercício

- Na figura (b) abaixo, se D requerer uma unidade a mais, isso levará a um estado seguro ou inseguro? E se a solicitação partir de C ao invés de D?

Has Max		
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

Has Max		
A	1	6
B	1	5
C	2	4
D	4	7

Free: 2

(b)

Has Max		
A	1	6
B	2	5
C	2	4
D	4	7

Free: 1

(c)

Prevenção de deadlocks

- As técnicas apresentadas para se evitar deadlocks são difíceis de serem implementadas por exigirem conhecimento sobre solicitações futuras
- A solução é evitar que pelo menos uma das quatro condições necessárias para ocorrência de deadlock ocorra, ou seja:
 - Atacando a condição de Exclusão mútua
 - Atacando a condição de Posse e espera
 - Atacando a condição de Sem preempção
 - Atacando a condição de Espera circular

Atacando a condição de exclusão mútua

- Se nenhum recurso fosse atribuído exclusivamente a um processo não haveria deadlocks
- Sem exclusão mútua pode-se criar o caos (ex.: acesso direto à impressora)
- A técnica de spooling para a impressora elimina o problema de exclusão mútua, pois só o “daemon” do spooler efetivamente acessa a impressora
- Problema é que nem todo processo pode usar a técnica de spooling
- **Recomendado:**
 - Somente atribuir recursos quando absolutamente necessário
 - Mínimo possível de processos devem ter privilégio para solicitar o recurso

Atacando a condição de posse e espera

- Pode-se eliminar o deadlock se um processo de posse de um recurso ficar impossibilitado de solicitar mais recursos
 - Processos requerem todos seus recursos antes de rodarem
 - Recursos disponíveis → processo roda por completo
 - Não disponíveis → processo espera
- Problemas
 - Total de recursos necessários só são sabidos qdo processo começa (→ Banker algoritmo seria usado)
 - Recursos não são usados otimizadamente (longas esp.)
- Variantes
 - Processo libera todos recursos em sua posse e, então solicita todos os recursos necessários de uma só vez

Atacando a condição de não preempção

- Solução pouco viável
- Certos recursos não podem ser removidos forçadamente sem prejuízos
- Ex.: uma impressora
 - É inviável parar uma impressão de um processo para começar a de outro processo !!!

Atacando a condição de espera circular

1. Imagesetter
2. Scanner
3. Plotter
4. Tape drive
5. CD Rom drive

(a)

se $i > j$, A não pode
Requerer j

Se $j > i$, B não pode
Requerer i

Então, loop é impossível



(b)



- Processos devem requerer recursos segundo uma ordem numérica pré-estabelecida
- Assim, os gráficos de recurso não têm loop
- pode ser difícil encontrar uma ordem que satisfaça a todos

Resumo da prevenção de deadlock

- Técnicas existentes

Condition	Approach
Mutual exclusion	Spool everything
Hold and wait	Request all resources initially
No preemption	Take resources away
Circular wait	Order resources numerically

Starvation

- Pode ocorrer em sistemas em que processos pequenos são escalonados prioritariamente
- Os processos longos podem ser adiados indefinidamente, embora não estejam bloqueados nem em deadlock
- Solução mais viável → escalonamento do tipo FIFO

Leitura Complementar

- ***Sistemas Operacionais Modernos, 2ª edição***
Andrew Tanenbaum, Editora Pearson, 2003.
- **Operating Systems (4th edition),**
William Stallings, Prentice Hall, 2001
- ***Sistemas Operacionais - Conceitos e Aplicações,***
A. Silberschatz, P. Galvin, e G. Gane, Editora Campus, 2001.
- **Sistemas Operacionais**
R. S. Oliveira, A. Carissimi e S. Toscani, Editora Sagra Luzzatto, 2004